ASSIGNMENT 1: DATA ENGINEERING

Apache Hive : Retail Transactions Analysis

Student Name	Diya S Ballal
USN	4GW23CI013
Branch	CSE(AI & ML)
Tool Covered	Apache Hive / HiveQL
Scenario	retail transaction analysis

1.SCENARIO & PROBLEM STATEMENT

A global retail company stores transaction records in flat CSV files on HDFS. The business intelligence team requires structured SQL access to analyze revenue patterns across store categories.

Create an external table in Hive mapping to raw transaction data stored on HDFS. Write HiveQL queries to clean missing data, aggregate total sales revenue, and calculate the average order value per product category.

2.ABOUT APACHE HIVE – THEORETICAL BACKGROUND

2.1. Apache Hive

Apache Hive is a data warehouse infrastructure built on top of Hadoop that provides a SQL-like interface, called HiveQL, for querying and managing large datasets stored in HDFS. Hive translates HiveQL queries into execution Job that run over the underlying storage, allowing users familiar with SQL to work with big data without writing low-level MapReduce code directly.

2.2. Apache Hadoop

Apache Hadoop is an open-source framework that enables distributed storage and processing of large datasets across clusters of commode ty hardware. It is built around two core components: the Hadoop Distributed File System (HDFS) for storage, and a processing engine (originally MapReduce, now commonly YARN-based engines) for computation.

2.3 HDFS (Hadoop Distributed File System)

HDFS is a distributed, fault-tolerant file system designed to store very large files across multiple machines. It follows a master-slave architecture consisting of a NameNode, which manages the file system namespace and metadata, and one or more DataNodes, which store the actual data blocks.

3. DOCKER-BASED ENVIRONMENT SETUP

To avoid the complexity of installing and configuring Hadoop and Hive manually, the entire environment was containerized using Docker Compose. Three services were defined: a NameNode, a DataNode, and a Hive server, all built from the official apache/hadoop and apache/hive images. The configuration below (docker-compose.yml) defines these services, their ports, volumes, and dependencies.

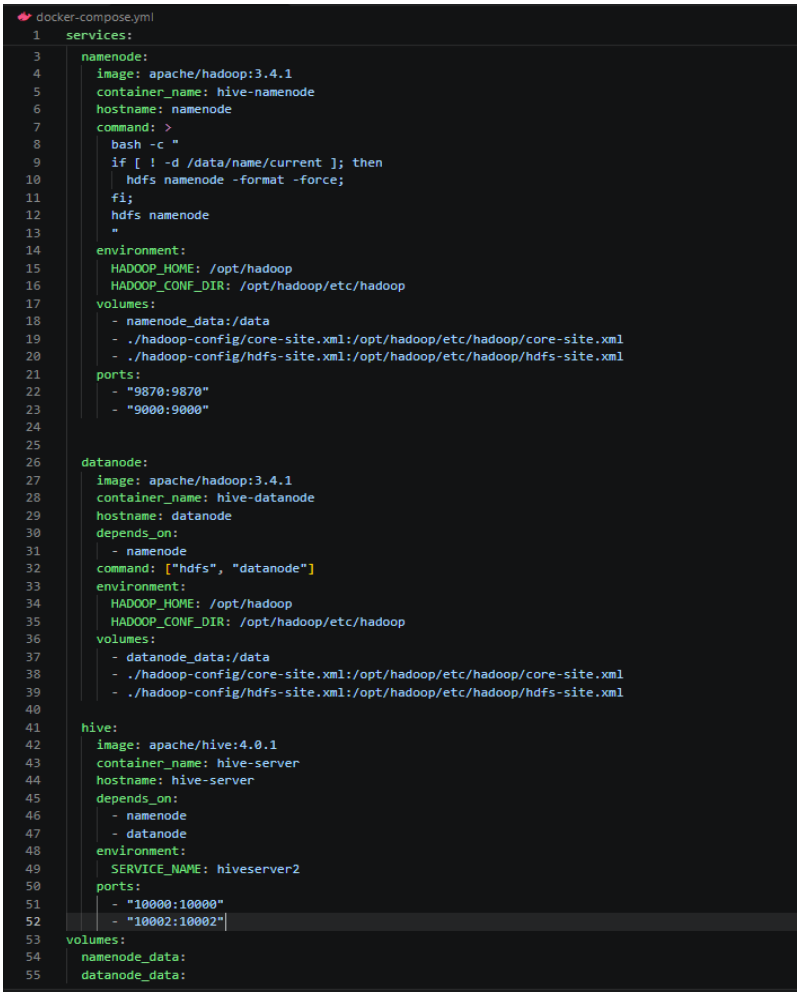


Figure 3.1: docker-compose.yml defining the namenode, datanode, and hive-server services.

The project directory contains the Docker Compose file, the Hadoop configuration XMLs (core-site.xml and hdfs-site.xml), and the raw dataset, as shown below.

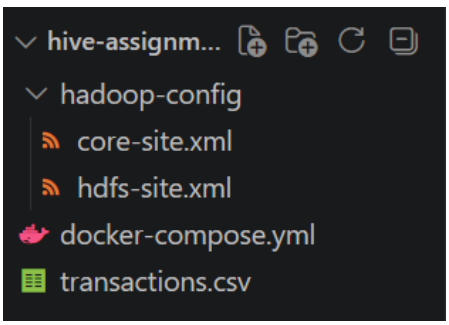


Figure 3.2: Project folder structure

The containers were started using the following command, which brings up all three services in detached mode:

```
docker compose up -d
```

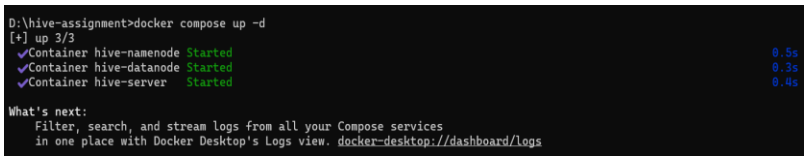


Figure 3.3: Output showing all three containers started successfully.

The status of the running containers was then verified:

```
docker compose ps
```



Figure 3.4: Output confirming hive-namenode, hive-datanode, and hive-server are all up.

Once the containers were running, a Beeline session was opened inside the Hive server container to interact with Hive via HiveQL:

```
docker exec -it hive-server beeline -u jdbc:hive2://localhost:10000
```



Figure 3.5: Beeline client successfully connected to the Hive server (Apache Hive 4.0.1).

4. DATASET DESCRIPTION

The dataset used for this assignment, transactions.csv, represents a small retail transactions log. It contains 15 rows and 5 columns: transaction_id, product_name, category, quantity, and price.

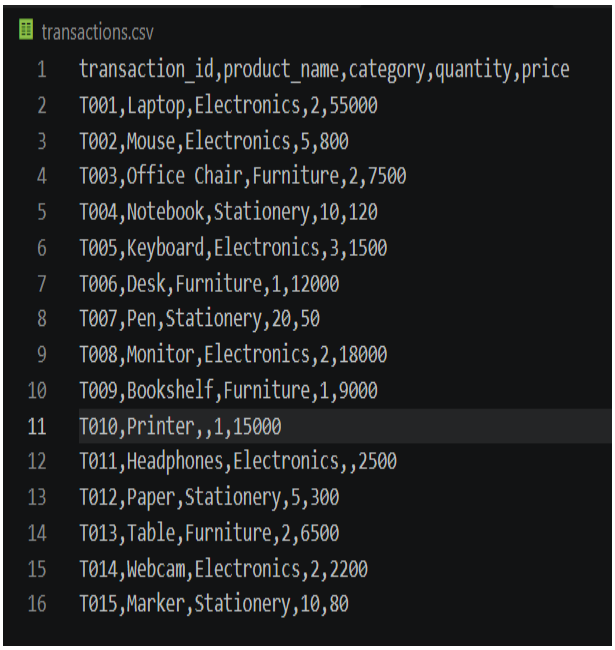


Figure 4.1: Raw transactions.csv dataset

Column	Type	Description
transaction_id	STRING	Unique identifier for each transaction
product_name	STRING	Name of the product purchased
category	STRING	Product category (Electronics, Furniture, Stationery)
quantity	INT	Number of units purchased
price	DOUBLE	Price per unit

5. HDFS DATA STORAGE

Before Hive can query the dataset, the raw CSV file must be placed into HDFS so that it is accessible to the distributed file system. The dataset was uploaded to the /retail_data directory on HDFS (mounted and referenced via the namenode service), which later becomes the LOCATION for the external Hive table. A fresh database, retail_db, was first created to logically group all objects for this assignment:

```
DROP DATABASE IF EXISTS retail_db CASCADE;

CREATE DATABASE retail_db;

SHOW DATABASES;

USE retail_db;
```



Figure 5.1: Successful creation of the retail_db database in Hive.

6. EXTERNAL HIVE TABLE CREATION

An external table named transactions was created over the CSV data stored in HDFS. Using an external table ensures that the underlying data in HDFS is preserved even if the table definition is dropped — only the metadata is removed, not the actual file. The table schema mirrors the CSV structure, with fields terminated by commas and the header row skipped using the TBLPROPERTIES setting.

```
CREATE EXTERNAL TABLE transactions (
    transaction_id STRING,
    product_name STRING,
    category STRING,
    quantity INT,
    price DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION 'hdfs://namenode:9000/retail_data'
TBLPROPERTIES ("skip.header.line.count"="1");
```

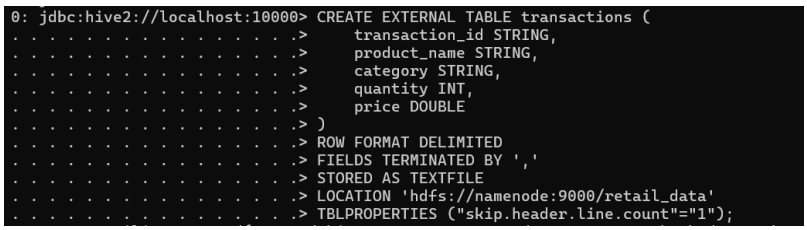


Figure 6.1: HiveQL statement creating the external 'transactions' table.

The table's existence and schema were then verified:

```
SHOW TABLES;

DESCRIBE transactions;
```

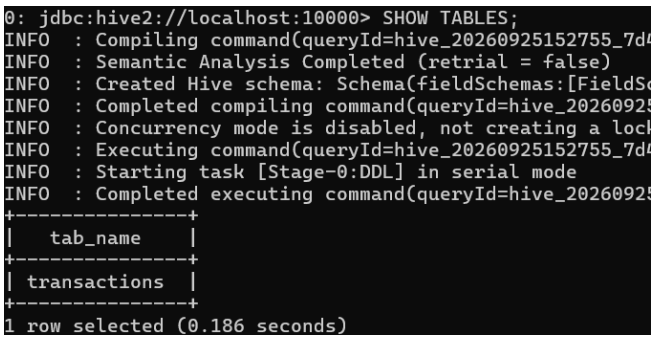


Figure 6.2: 'SHOW TABLES' confirming the 'transactions' table exists in retail_db.

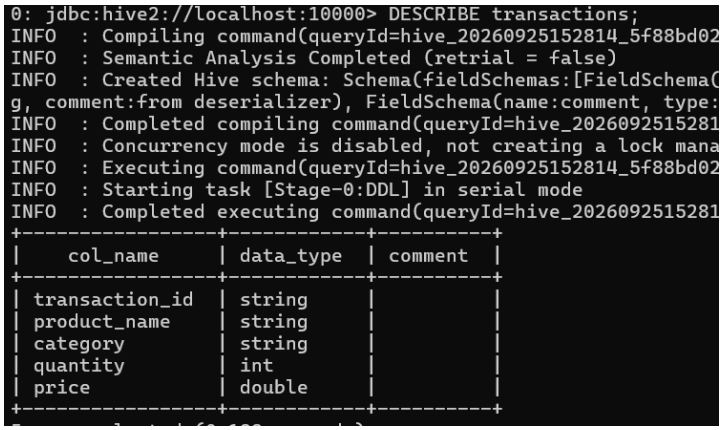
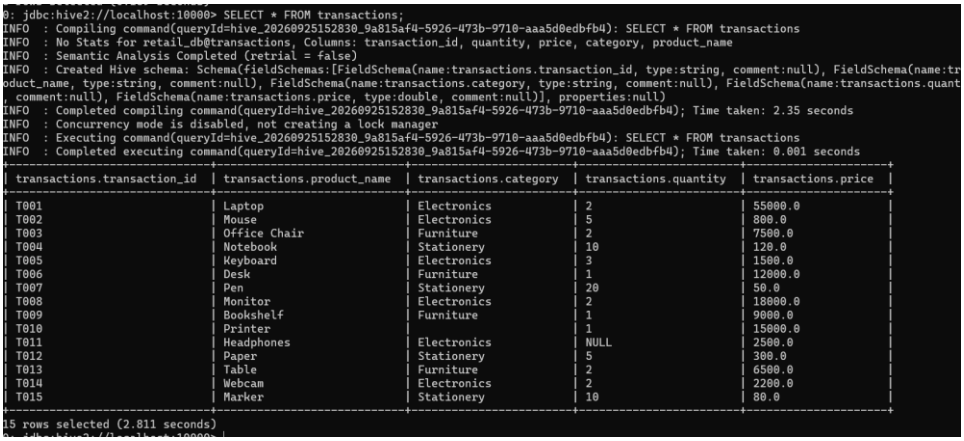


Figure 6.3: 'DESCRIBE transactions' showing the table schema with correct column types.

```
SELECT * FROM transactions;
```



7. MISSING DATA IDENTIFICATION

Before any aggregation could be trusted, the dataset was checked for missing or blank values. The query below identifies rows where the category is NULL or an empty string, or where the quantity is NULL:

```
SELECT *
FROM transactions
WHERE category IS NULL
   OR TRIM(category) = ''
   OR quantity IS NULL;
```

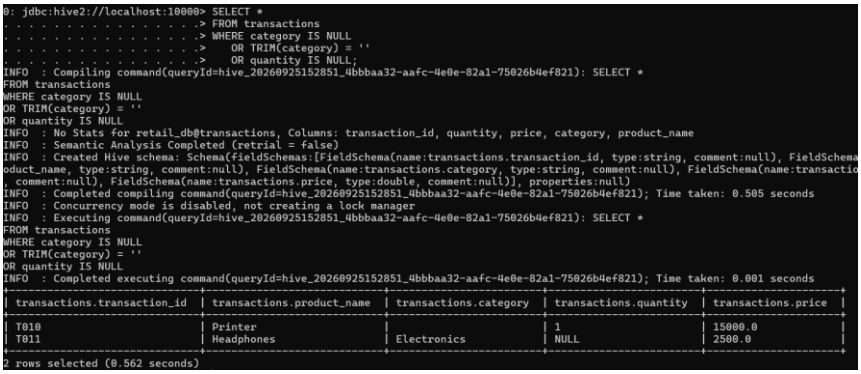


Figure 7.1: Two problematic rows identified — T010 with a missing category and T011 with a missing quantity.

8. DATA CLEANING

A new table, cleaned_transactions was created.

```
CREATE TABLE cleaned_transactions AS
SELECT
    transaction_id,
    product_name,
    CASE
        WHEN category IS NULL OR TRIM(category) = '' THEN 'Unknown'
        ELSE category
    END AS category,
    CASE
        WHEN quantity IS NULL THEN 1
        ELSE quantity
    END AS quantity,
    price
FROM transactions;
```

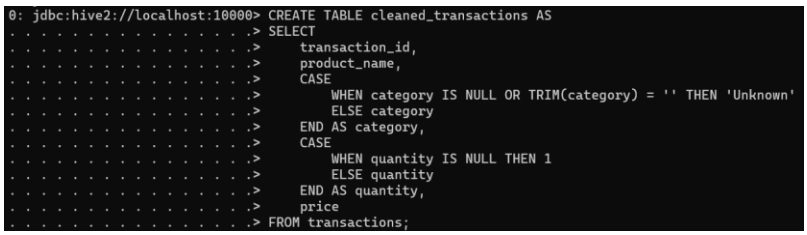


Figure 8.1: cleaned_transactions table.

The cleaned table was then verified in full, confirming that T010's category was replaced with 'Unknown' and T011's quantity was replaced with 1, while all other rows remained identical to the source data.

```
SELECT * FROM cleaned_transactions;
```

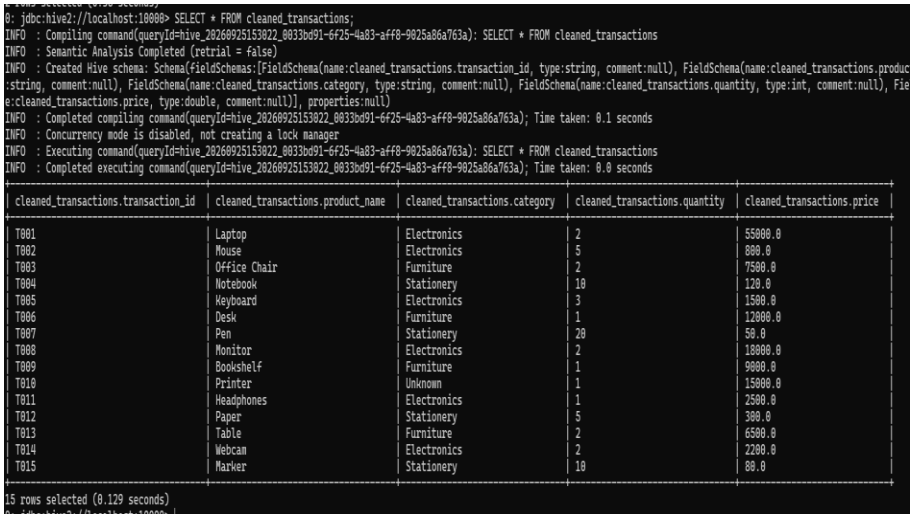


Figure 8.2: cleaned_transactions table — all 15 rows now free of missing values.

9. TOTAL SALES REVENUE

With the cleaned dataset in place, the total sales revenue was calculated by multiplying quantity and price for every record and summing the results across the entire table:

```
SELECT
    SUM(quantity * price) AS total_sales_revenue
FROM cleaned_transactions;
```

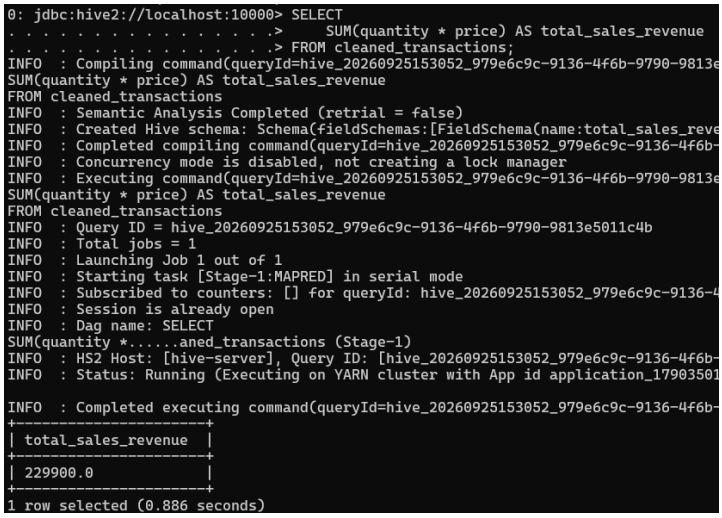


Figure 9.1: Total sales revenue computed as 229,900.0.

10. AVERAGE ORDER VALUE BY CATEGORY

To understand spending patterns across product categories, the average order value (quantity × price) was computed for each category using a GROUP BY aggregation:

```
SELECT
    category,
    AVG(quantity * price) AS average_order_value
FROM cleaned_transactions
GROUP BY category;
```

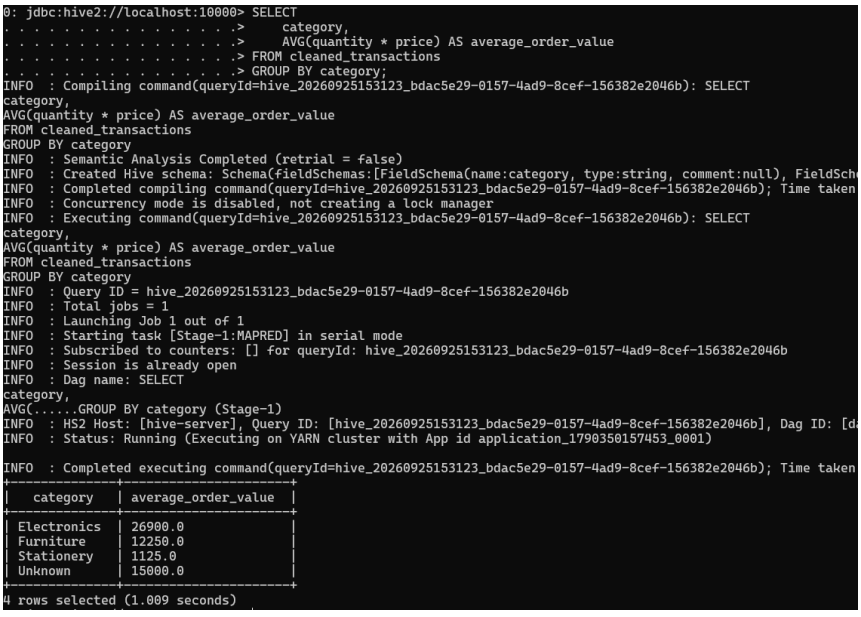


Figure 10.1: Average order value grouped by category.

11. CONCLUSION

- This assignment provided hands-on experience with the core components of the Hadoop ecosystem — HDFS for distributed storage and Apache Hive for SQL-like querying — deployed through a fully containerized Docker environment.
- The workflow covered environment setup, external table creation over HDFS-resident data, systematic identification and cleaning of missing values, and the computation of meaningful business metrics such as total revenue and average order value by category.
- These skills form a foundational part of the data engineering stage of an MLOps pipeline, where raw, imperfect data must be reliably transformed into clean, analysis-ready datasets before any downstream modeling or reporting can take place.